# Explainable Artificial Intelligence-Based Wafer Map Defect Diagnosis and Decision Support System

Final Project Report

Data Analysis and Machine Learning with Python

**Authors**

Abhirup Sain (T14H06318)
David Spickenheuer (T14H06329)
Martin Werner (T14H06319)
Matti Lehmann (T14H06309)
Staniya Thomas (T14H06310)

# Abstract

Every electronic device you own whether a smartphone or a car's control unit depends on microchips. Those chips are made in bulk on thin circular discs of silicon called wafers, where hundreds of identical chips are etched side by side. Before a wafer leaves the factory, every single chip is electrically tested, and the results are recorded as a color-coded grid image: green for a chip that passed, red for one that failed. Engineers call these images wafer maps, and the patterns of failures they reveal are like fingerprints. A ring of failures around the edge points to one kind of manufacturing problem, a streak across the middle points to another, a scattered cluster of failures points to something else entirely. Identifying the right pattern quickly is critical, because the answer determines whether a whole production batch gets scrapped, whether a piece of equipment gets pulled offline for maintenance, or whether everything just keeps running. Getting it wrong wastes millions worth of materials and machine time; getting it slowly is nearly as costly.

This project set out to build a tool that could actually help with that decision. We developed an explainable AI-based wafer map defect diagnosis system that combines a U-Net-inspired segmentation model (about 3.2 million parameters) with a spatially-attentive CNN classifier (around 2.1 million parameters). Together they form a two-stage pipeline: the segmentation model first highlights which parts of the wafer correspond to each defect type, then the classifier reads those spatial maps and produces a confidence score for each of the eight defect categories in the WM-811K dataset. Engineers can interact with the system either by uploading a wafer image or by sketching a pattern freehand on a canvas in the Streamlit interface.

A few design choices shaped the system in ways worth flagging upfront. We chose a two-stage pipeline over a simpler end-to-end classifier because we wanted the model to show its work, spatial activation maps are themselves interpretable, not just a by-product. We added a custom Spatial Attention layer (drawn from the CBAM paper) to each convolutional block in the classifier, so the network concentrates on the small regions where defects actually live rather than treating the whole wafer equally. And because the WM-811K dataset only provides single-defect labels, we created synthetic multi-defect training samples by compositing pairs of single-defect wafer maps, which lets the system handle the real-world situation where two defect types appear on the same wafer.

Training went through three phases: segmentation model training alone (converging to MSE loss near 0.017 after 100 epochs), classification head pre-training with the segmentation model frozen (reaching AUC 0.93 to 0.94 at best epoch 19 of 20), and end-to-end fine-tuning of the full pipeline (reaching AUC above 0.96 at best epoch 40 of 50). The segmentation output visually confirms that the model has learned meaningful spatial structure in which defect-specific regions light up correctly while unrelated channels stay dark.


# Dataset

### WM-811K: Where the Data Comes From

The WM-811K dataset is the empirical foundation of this project. It contains 811,457 real wafer maps collected from semiconductor fabrication lines, each stored as a 2D array of integers with pixel values in the set {0, 1, 2}: 0 for background (outside the circular wafer

boundary), 1 for a passing die, and 2 for a failing die. The dataset covers eight named defect classes namely Center, Donut, Edge-Loc, Edge-Ring, Loc, Random, Scratch, and Near-full plus a None class for defect-free wafers.

Two characteristics of the raw dataset make it genuinely challenging to work with. First, the wafer maps come in many different sizes and aspect ratios, because they were captured from different tools and fabs over time, a map from one tool might be 14×14 pixels while another might be 200×190. Any model that takes fixed-size input needs to handle this variability consistently. Second, and more strikingly, the vast majority of the dataset has no label at all. Of the 811,457 wafer maps, only 12,822 carry a defect class annotation less than 1.6% of the total. All supervised training in this project was performed on the labelled subset only.

### Preprocessing the Wafer Maps

Before any wafer map can be fed to a model, three preprocessing steps are applied consistently. First, all maps are resized to a fixed target resolution of 64×64 pixels using nearest-neighbour interpolation, with padding added first to preserve the original aspect ratio so circular wafer maps are not stretched into ellipses. Second, pixel values are rescaled from raw {0, 1, 2} integers to normalized {0.0, 0.5, 1.0} floats by dividing by 2.0. Third, data augmentation is applied during training with random horizontal and vertical flips and random 90-degree rotations to expand the effective training set and reduce overfitting. Augmentation is only applied at training time; inference uses the preprocessed image directly without augmentation.

### Creating Multi-Defect Training Samples

WM-811K provides only single-defect labelled samples whereas wafers in practice can exhibit two or more defect types simultaneously. We generated synthetic multi-defect training samples by compositing pairs of single-defect maps. Two maps are selected from different defect classes, their pixel arrays combined using an element-wise maximum so that failing dies from either source are preserved, and their one-hot label vectors merged with a logical OR to produce a multi-hot label marking both classes as present. This approach is cheap to produce, requires no expert annotation, and generates plausible composites for most class pairs. Its main limitation is the additive assumption, it works well when the two defect patterns occupy different spatial regions but is less realistic when they heavily overlap.

### The WM_811K Dataset Class

All dataset logic is centralized in a single Python class, WM_811K, which acts as the sole source of truth for every dataset-related constant and operation across the project. The class holds the label-to-integer mapping, the target image size (64×64), and static methods for preprocessing and augmentation. Because model architectures, training scripts, and the Streamlit UI imports from this one class, any change propagates everywhere automatically with no risk of silent inconsistencies between files.

The class exposes three TensorFlow dataset pipelines: dataset_single_defect() for training on individual defect classes, dataset_multi_defect_segmentation() for training the segmentation model with per-class binary mask targets, and dataset_multi_defect_fullstack() for the end-to-end fine-tuning phase. Each pipeline handles batching, shuffling, and augmentation internally.

## I. Application Context and Decision Problem

## System Application Scenarios

At the end of the wafer probe test step, every die on a wafer is electrically tested and the result (pass or fail) is stored as a pixel in a 2D binary image. The pixel values follow a simple convention: 0 for background (outside the wafer boundary), 1 for a passing die, and 2 for a failing die. When a batch of wafers is complete, these maps get reviewed to work out what went wrong and where. The WM-811K dataset catalogues eight distinct defect patterns that process engineers commonly encounter Center, Donut, Edge-Loc, Edge-Ring, Loc, Random, Scratch, and Near-full alongside a None class for wafers where everything looks clean. Out of the full 811,457 wafer maps in the dataset, only 12,822 actually carry a defect label, which says something about how sparse real annotation effort tends to be in practice.

We designed the system with two concrete use cases in mind. The first is real-time inspection: an engineer uploads a freshly generated wafer map and wants an instant read on what defect type, if any, is present. The second is hypothesis testing: an engineer has a theory about what they are seeing across a batch and wants to sketch the suspected pattern on the canvas to see whether the model agrees a surprisingly useful way to sanity-check a diagnosis before escalating.

## Decision-Making Challenges Faced by Users

- Visual ambiguity: Edge-Loc and Edge-Ring both cluster failures near the wafer perimeter, are difficult to distinguish by eye, especially under fatigue.
- Co-occuring Defects: Wafers frequently exhibit multiple defect types simultaneously, making it hard to untangle separate root causes manually.
- High Production Volume:  A modern fab can produce thousands of wafers a day; there is simply no bandwidth to hand-examine every map.
- Lack of Spatial Insight: Traditional tools classify the defect type but fail to localize where it occurred, delaying the physical tracing of tool faults.

## Limitations of Existing Approaches

Most automated defect classification tools in production today apply a single-label classifier and hand back a category. They do not handle multiple co-occurring defect types, they do not show which spatial region drove the prediction, and they offer no intermediate representation that could help an engineer understand or challenge the result. Academic research has pushed accuracy on WM-811K quite high, but the best-performing models tend to be large CNNs that behave as black boxes. Post-hoc methods like Grad-CAM can generate a rough heatmap, but it is a single blended overlay not the per-class, per-region breakdown that would actually be useful at a review station.

## Specific Decisions Supported by the System

At its core the system is trying to answer two questions an engineer would naturally ask. First: which defect type or types are present, and with how much confidence? That is answered by the sigmoid probability scores from the classification head, one per defect class. Second: where on the wafer is each defect showing up? That is answered by the eight spatial activation maps produced by the segmentation model, each one highlighting the regions most associated with a particular defect pattern. Together, those two outputs give an engineer something much closer to the explanation they actually need.

# II. Related Design Approaches

## Design of Related Systems

Previous attempts have fallen into one of three camps. The oldest approach is rule-based template matching, where wafer maps are compared against a library of hand-crafted spatial templates. It is transparent and fast, but it falls apart the moment a defect looks slightly different from what is in the template library, and it has no concept of multiple defects appearing together. The second camp uses classical machine learning SVMs, random forests and operating on hand-crafted features like die density, centroid position, and region geometry. These methods are more flexible but require a lot of domain expertise to engineer good features, and they tend to be brittle when fab conditions shift. The third camp, applies deep CNNs directly to the raw 2D wafer map. These reach impressive accuracy on the WM-811K benchmark, and multi-label variants using sigmoid outputs and binary cross-entropy loss handle co-occurring defects reasonably well but lacks explainability. Grad-CAM patches over this partially, producing a rough heatmap after the fact, but it is an approximation and it produces one blended map rather than separate channel-by-channel localization.

## Limitations in This Context

- Template matching cannot adapt to defect morphologies it has not seen before, and the multi-defect case is essentially outside its scope.
- Classical ML pipelines need domain experts to design features, and those features often stop working well when the process environment changes.
- Deep CNN classifiers, even with Grad-CAM bolted on, give you a single mixed-class heatmap rather than spatially separated per-defect explanations.
- None of these approaches let an engineer interact with the system by drawing as they are all passive classifiers, not tools for exploration.

## Differences From Prior Approaches

What we have tried to build here is different in a few meaningful ways. The per-class spatial activation maps are a primary output, they come directly from the segmentation model's decoder rather than being approximated post-hoc. The multi-label capability is baked in from the training data design rather than being an architectural add-on. And the whole thing is wrapped in an interface that lets engineers engage with it interactively, which changes how you use a tool like this in practice.

# III. Artifact Overview

## Overall System Functions

The system is built from four pieces that fit together into a coherent pipeline.

- WM_811K Dataset Module: This is the data foundation. It loads the raw wafer map data from the .pkl files, exposes three TensorFlow dataset pipelines (single-defect, multi-defect segmentation, and multi-defect full stack), and acts as the single source of truth for image dimensions, label-to-integer mappings, and augmentation logic. Having

everything defined in one place meant that when we changed the image size during development, nothing broke downstream.

- Segmentation Model (U-Net-inspired, ~3.21 M parameters): The encoder passes the input through four convolutional blocks at filter depths 32, 64, 128, and 256, with Max Pooling to progressively reduce spatial dimensions. The bottleneck uses Global Average Pooling followed by two Dense layers, then reshapes and up samples back before feeding into the decoder. Skip connections from the encoder let the decoder recover spatial detail that would otherwise be lost in the bottleneck. The output is an 8-channel map at 96×96 resolution, one channel per defect class.
- Classification Model (CNN + Spatial Attention, ~2.11 M parameters): This model takes the 8-channel segmentation output as its input. Each of its four convolutional blocks is immediately followed by a custom Spatial Attention layer that computes a spatial mask from average- and max-pooled channel features and uses a 7×7 convolution to decide where on the feature map to focus. After Global Average Pooling and two Dense (512) layers with Batch Norm and Dropout, it outputs 8 sigmoid probabilities, one per defect class.
- Streamlit User Interface (app_best_wafer_cnn.py): The front end is a polished Streamlit web application that loads the trained best CNN model (best_wafer_cnn.keras) and offers two input modes. In Upload mode, engineers drag in a wafer map image; in Draw mode, they sketch directly on a freehand canvas. Either way, the image gets preprocessed and passed through the model, and the results come back as a 3x3 grid of confidence cards each showing the defect name, a percentage, and a colour-coded progress bar. A second interface (app_direct_classifier_cnn.py) provides the same experience wired to the direct CNN classifier, serving as the comparison baseline in the UI evaluation.

## System Boundaries

The system works with 2D wafer map images (PNG, JPG, or JPEG) and freehand canvas drawings. It does not touch raw electrical test data, lot-level metadata, or SPC control charts. It does not write anything to a database or trigger any downstream actions. Its role is entirely advisory, it offers a diagnosis, and the engineer decides what to do with it.

## Usage Workflow

- Input: The engineer either uploads a wafer map file or draws a pattern on the interactive canvas.
- Preprocessing: The image is converted to grayscale, pixel values are normalized to the set {0, 1, 2} by rounding pixel/127.5, and the image is resized to 64×64 with padding to preserve the aspect ratio.
- Inference: The preprocessed image goes through the segmentation model, producing eight spatial activation maps. Those maps are then passed to the classification model, which outputs a confidence score for each defect class.
- Results: The confidence cards appear in a 3×3 grid. Bars are highlighted in cyan/blue when confidence is above 70%, grey when it is below 30%, with a gradient in between.

# IV. Design Decisions and Trade-offs

## Decision 1: Two-Stage Segmentation + Classification vs. Direct End-to-End Classification

**Why:** Provides clear visual interpretability (e.g., lighting up the wafer periphery for an Edge-Ring), unlike the simpler end-to-end model which is difficult to debug when it misclassifies. We also built app_best_wafer_cnn.py, a standalone Streamlit interface wired to the best CNN model directly, for comparison. While it is simpler to train and deploy, but hard to understand why when something goes off.

**Trade-off:** The two-stage approach means more work upfront. The segmentation model has to be trained to a reasonable level before the classifier pre-training even starts. The pipeline is also more complex to debug when both components are unfrozen during fine-tuning.

**Best for:** This design makes the most sense when spatial interpretability matters which, for process fault diagnosis, it almost always does. For a simpler task where you just need a fast binary answer and do not care about the spatial explanation, the direct classifier would be the better choice.

## Decision 2: Spatial Attention in Every Convolutional Block

**Why:** Wafer defects are often tiny relative to the full wafer area. A scratch defect might cover less than 5% of the image. If the network treats every spatial location equally, it wastes a lot of capacity modelling background. The Spatial Attention layer inspired by the CBAM paper generates a soft mask by pooling feature maps across channels and running a 7×7 convolution, then multiplies that mask back onto the feature maps. In practice this steered the network toward the regions that actually matter.

**Trade-off:** Channel attention (SE-Net style) was an alternative, which lacks the geometric and geographical focus needed for spatial defect tracking.

**Best for:** Spatial attention is most valuable when defects are small and localized. For defects like Near-full, which cover most of the wafer, it is less critical but it does not hurt either.

## Decision 3: Synthetic Multi-Defect Sample Generation

**Why:** WM-811K only has single-defect labels. In the real world, two process problems can hit the same lot simultaneously, producing a wafer map with both a random scatter and an edge ring. We had to either ignore multi-defect scenarios or create training data for them ourselves. We created synthetic composites by taking two single-defect maps, blending them with an element-wise maximum, and combining their one-hot labels with a logical OR. It is not perfect, but it gave the model exposure to multi-label scenarios without requiring any additional human annotation.

**Trade-off:** Element-wise maximum blending is a simplification. Real co-occurring defects may interact spatially in ways our synthetic composites do not capture. For example, a scratch that cuts across a pre-existing edge ring region. This is a known limitation and something we would address with real labelled multi-defect data if it were available.

**Best for:** This approach works well when defect patterns are spatially non-overlapping or only marginally overlapping. Heavily overlapping defects (like Donut and Near-full, which both cluster near the wafer edge) are the hardest cases for the synthetic approach.

### Decision 4: Three-Phase Training Curriculum

**Why:** Early in the project we tried training the whole pipeline end-to-end from scratch. The classification head was receiving segmentation outputs that were essentially random noise in the first few epochs, which made gradient flow erratic and slowed everything down. Breaking training into three phases (1) train the segmentation model alone, (2) freeze it and pre-train the classification head, (3) unfreeze everything for joint fine-tuning. Let each component get its bearings before being asked to work with the other. The results were noticeably cleaner.

**Trade-off:** Phase-by-phase training is more involved. You have to monitor three separate training runs, pick checkpoints at phase transitions, and be careful about learning rate settings when you unfreeze the backbone. We used Model Checkpoint and Early Stopping callbacks throughout (patience 10–15 depending on the phase), which helped manage this, but it still required more hands-on attention than a single training loop would.

**Best for:** Curriculum training is most useful when one component's output is another component's input and the upstream model needs time to stabilize before the downstream model can learn anything meaningful. It is a broadly applicable pattern for any multi-stage deep learning pipeline.

## V. AI Role and System Autonomy

### The Role of AI in the System

The segmentation model acts as an analyst: it looks at a raw wafer map and produces a spatial breakdown of where each defect type appears, effectively translating a single greyscale image into eight overlay maps that highlight what is going on where. The classification model then acts as an interpreter: it reads those spatial maps and converts them into calibrated probabilities for each defect class. Neither model makes a decision but they produce information that supports a decision.

### Level of System Automation

Once an image is provided, the preprocessing, segmentation, and classification all happen automatically with no further input from the user. But that is where the automation stops. The system does not flag a wafer for scrap, raise a process alarm, notify a supervisor, or update any external database. Every action downstream of the prediction remains entirely with the engineer. This was a deliberate choice and not a technical limitation. Automating the response to a defect classification requires a level of reliability and accountability that goes well beyond what a research prototype should be claiming.

### How Users Interact with and Oversee AI

Before inference, Engineers control the inputs by uploading real wafer maps or sketching custom patterns, transforming the system into an active tool for exploratory testing. After inference, the engineer reviews the confidence card display and, if they want to dig deeper, can inspect the segmentation model's per-class activation maps to see which regions drove the high-confidence calls. Mismatches between high confidence and vague activation maps prompt healthy skepticism, encouraging critical human oversight.

# VI. Evaluation System, Logic and Evidence

## Evaluation Goals and Methods

We had three main questions going into the evaluation. Did the segmentation model actually learn to localize defect regions, or was it just producing plausible-looking noise? Did the classification model reach a level of AUC that would make it useful in practice? And does the two-stage pipeline produce meaningfully better or more interpretable results than the simpler direct classifier? We did not run a formal user study, so the evaluation is quantitative (training metrics) and qualitative (visual inspection of segmentation outputs).

## Evaluation Scenarios and Baselines

For the segmentation model, we compared the predicted per-class activation maps side by side with the ground-truth defect masks from the WM-811K dataset. For the classification model, we tracked multi-label AUC and binary cross-entropy loss across training epochs, with a held-out validation split. For the pipeline comparison, we ran the full stack model alongside the best CNN classifier (app_best_wafer_cnn.py) and the direct CNN classifier (app_direct_classifier_cnn.py) on the same test inputs and compared confidence scores and spatial outputs. These two standalone classifiers are the simpler designs that we actually built and deployed, not hypothetical alternatives.

## Performance and Results

### Phase 1 : Training the Segmentation Model

We trained the segmentation model for 100 epochs at 64 steps per epoch with a batch size of 16. The loss started at 0.140 in epoch 1 and dropped sharply over the first five epochs before settling into a long, gradual decline to about 0.017 by epoch 100. The curve is smooth enough to confirm stable convergence without early stopping triggering.

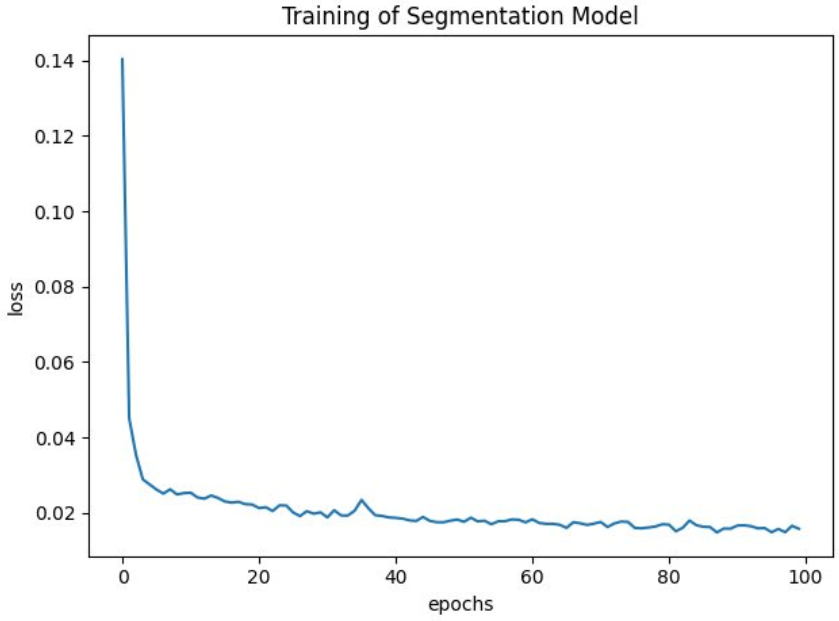


Figure 1. Segmentation model training loss over 100 epochs (MSE). The sharp drop in the first few epochs reflects rapid learning of coarse spatial structure; the slow decline afterward reflects fine-grained refinement.

The visual comparison tells the more interesting story. Before training, the model outputs undifferentiated noise across all eight channels and it has no spatial preference. After 100 epochs, the per-class maps are clearly structured: the Center channel activates near the middle of the wafer, the Edge-Ring channel shows a circular ring around the periphery, the Scratch channel traces a linear streak, and channels corresponding to defect types not present in the sample stay near zero. That is exactly the behaviour we were aiming for.

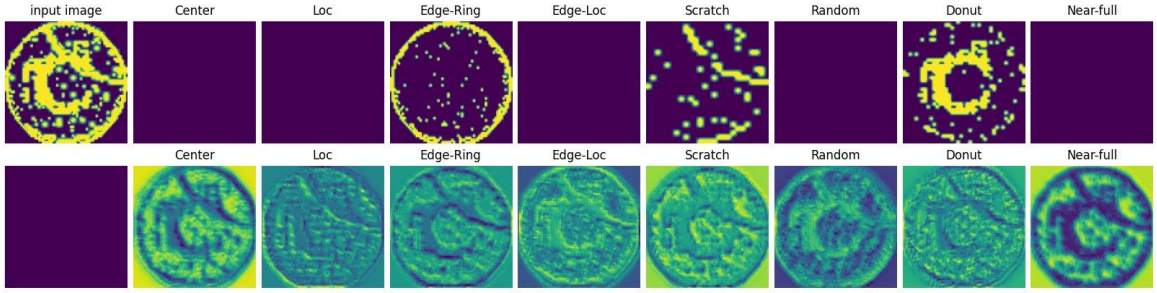


Figure 2. Segmentation model output before any training. Top row: ground-truth per-class defect masks. Bottom row: model predictions structureless noise with no spatial coherence across any of the eight channels.

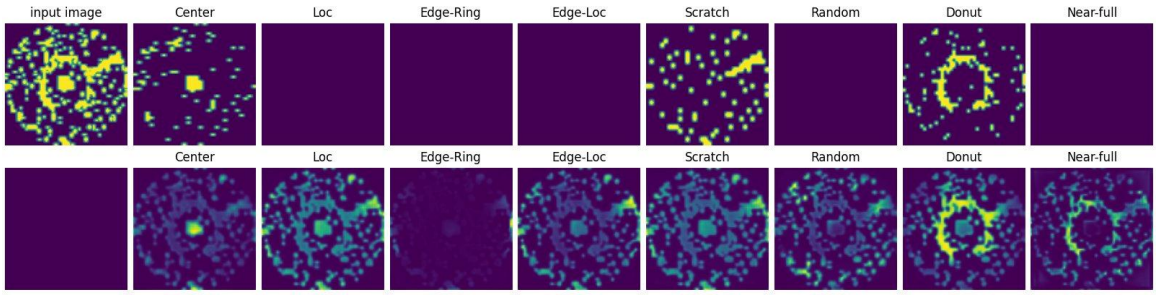


Figure 3. Segmentation model output after 100 epochs. The model now correctly localizes Center, Edge-Ring, Scratch, and Donut regions in their respective channels while keeping unrelated channels suppressed. The spatial structure matches the ground-truth masks well.

## Phase 2: Pre-Training the Classification Head

With the segmentation model frozen, we pre-trained the classification head on its outputs. Figure 4 shows the AUC and loss curves over approximately 32 epochs. AUC climbs from 0.65 to 0.86, and binary cross-entropy loss falls sharply from 0.82 in the first five epochs down to around 0.35, then continues gradually toward 0.30. The clean monotonic loss curve confirms that the classification head is learning steadily without instability which is a good sign that the segmentation outputs it is consuming are consistently structured. The refined train/validation split curves are shown in Figure 5.

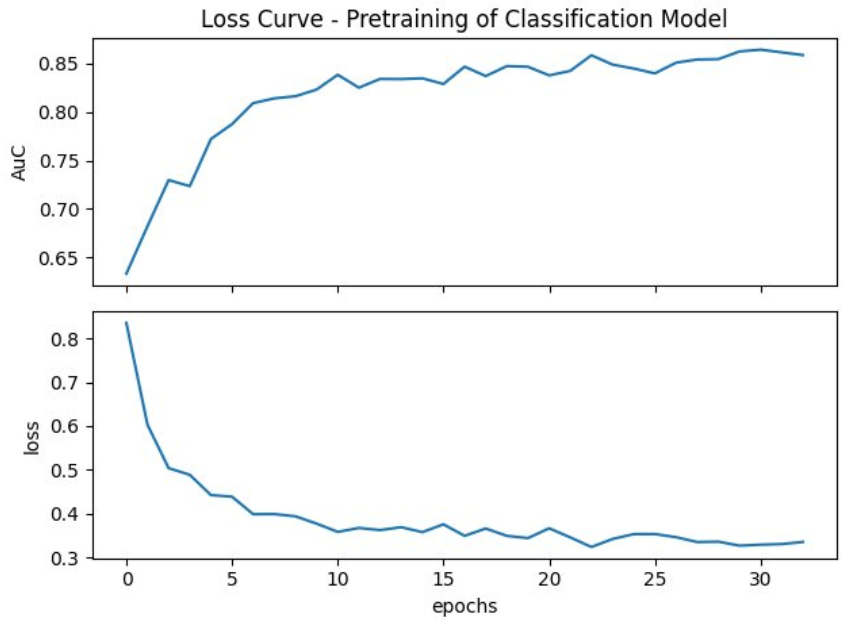


Figure 4. Phase 2 classification head pre-training curves (segmentation model frozen, approximately 32 epochs). Top: AUC rises from 0.65 to 0.86. Bottom: binary cross-entropy loss falls from 0.82 to approximately 0.30, confirming stable learning from frozen segmentation outputs.

## Phase 2 Refined: Classification Head with Train/Validation Tracking

Figure 5 shows the full Phase 2 training curves with both training and validation tracked. AUC climbs steadily from 0.81 to 0.93 on the training set, with validation closely following and reaching 0.94 at best epoch 19. The loss curves converge in parallel near 0.23, with no meaningful gap between training and validation, a reassuring sign that the head is learning generalizable patterns from the segmentation outputs rather than memorizing the training set.

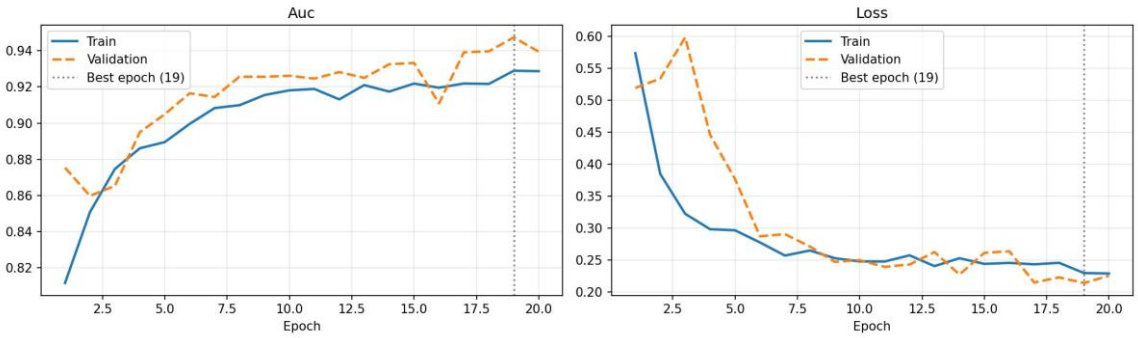


Figure 5. Phase 2 refined training curves (20 epochs, segmentation model frozen). Left: multi-label AUC — training reaches 0.93, validation 0.94 at best epoch 19. Right: binary cross-entropy loss — both training and validation converge near 0.23 with no overfitting.

## Phase 2 Extended: Observing Convergence Behavior Over 50 Epochs

Before moving on to full fine-tuning, we ran the classification head for a longer 50-epoch window to observe how the model behaves when given more time with the segmentation backbone still frozen. Figure 6 shows the result. Training AUC starts around 0.91 and rises steadily to approximately 0.96 by epoch 50, while validation AUC is more volatile peaking

above 0.98 at best epoch 47. Training loss falls from 0.27 toward 0.175, and validation loss trends down to a minimum near 0.15. The higher validation volatility compared to the short 20-epoch run in Figure 5 is expected over a longer training window, but the overall downward trend confirms the classification head is continuing to improve without overfitting. Best epoch 47 was saved as the checkpoint carried into Phase 3.

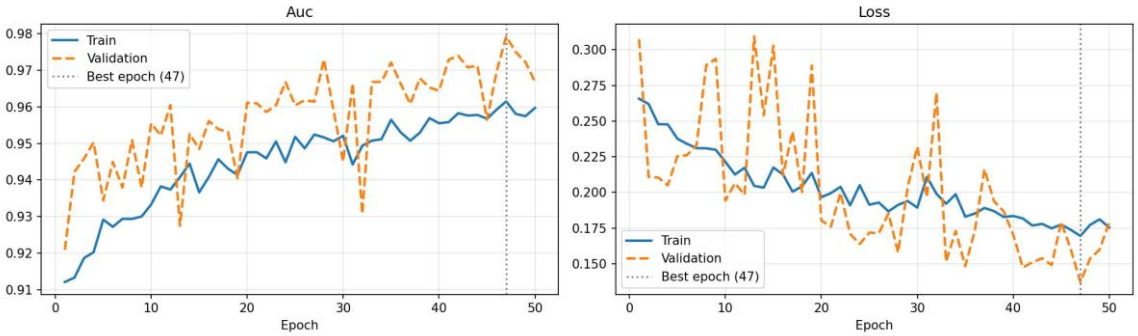


Figure 6. Phase 2 extended training curves over 50 epochs with the segmentation model still frozen (best epoch 47). Left: training AUC rises to 0.96 and validation AUC peaks near 0.98. Right: training loss falls to approximately 0.175 and validation loss reaches a minimum near 0.15, with higher epoch-to-epoch variance than the short Phase 2 run due to stochastic mini-batch sampling over the extended window.

## Phase 3 : End-to-End Fine-Tuning

For Phase 3 we unfroze the entire pipeline and fine-tuned end-to-end, starting from the Phase 2 checkpoint. Training AUC rises from 0.91 to around 0.95 by the end of the run, with validation AUC tracking closely and peaking above 0.96. The best checkpoint was saved at epoch 40. Training loss falls steadily from 0.27 toward 0.20, and validation loss follows a similar downward trend. Compared to Phase 2 Extended in Figure 6, the validation curve here is notably smoother which may seem counterintuitive since the backbone is now unfrozen, but reflects that the Phase 3 run used a larger batch size of 64, which averages out more gradient noise per update than the smaller batches used in pre-training.

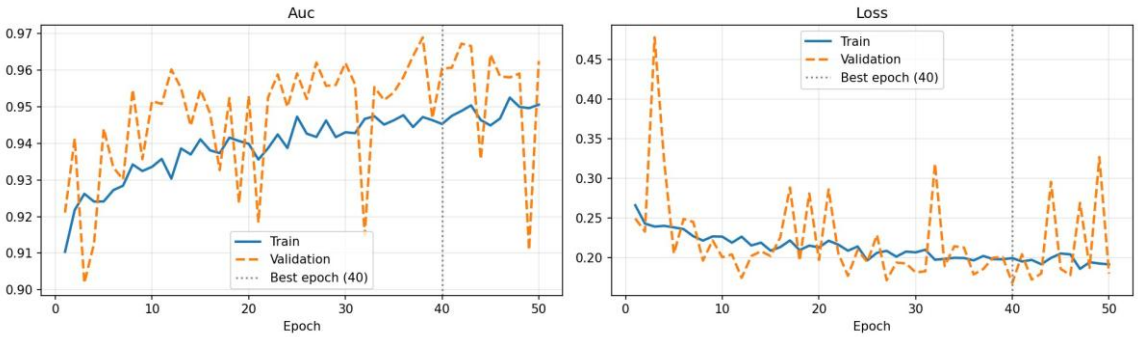


Figure 7. Phase 3 end-to-end fine-tuning curves with all layers unfrozen (best epoch 40). Left: training AUC rises to approximately 0.95 with validation peaking above 0.96. Right: training and validation loss both decline steadily toward 0.20, showing stable joint optimization of the segmentation backbone and classification head.

## Full stack Model Fine-Tuning Loss Curve

Figure 8 shows the AUC and loss curves specifically for the full stack model fine-tuning phase, plotted over 10 epochs to show the early convergence behavior in detail. AUC starts at 0.61

and rises to approximately 0.70 over the run. Loss begins at 0.60 and falls steadily to around 0.45, with a brief plateau around epoch 6 before continuing to improve. This shorter fine-tuning window captures the critical early phase where the segmentation and classification components begin to jointly adapt the steady loss drop without spikes confirms that the learning rate and batch size settings were well-suited for this stage.

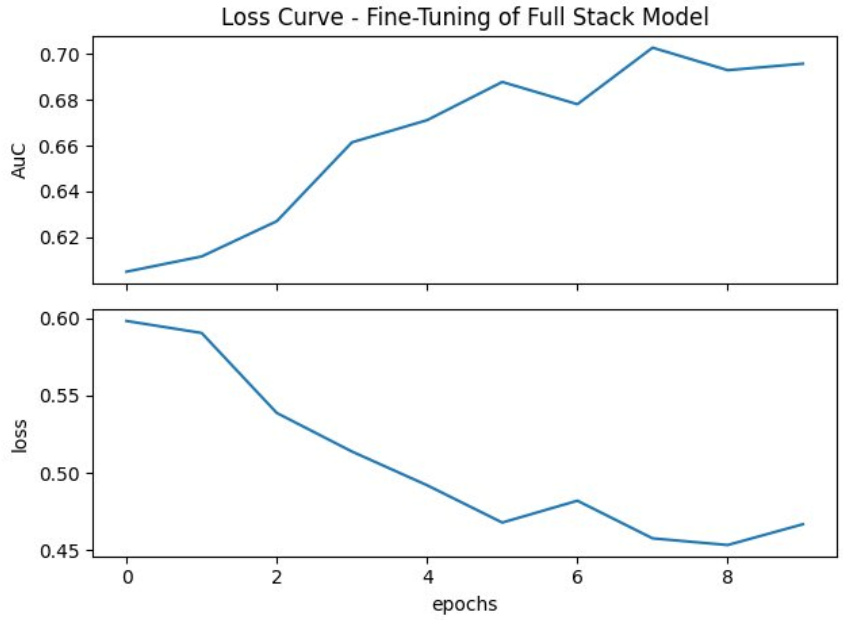


Figure 8. Full stack model fine-tuning loss and AUC curves (10 epochs). Top: AUC rises from 0.61 to approximately 0.70. Bottom: binary cross-entropy loss falls from 0.60 to approximately 0.45, showing stable early convergence of the jointly trained pipeline.

## Segmentation Model Output After Training

Figure 9 shows the segmentation model's per-class activation maps on a real test wafer after the full training pipeline. The input wafer map (top left) contains an Edge-Ring pattern with a scattered Scratch overlay. The top row shows the ground-truth per-class binary masks, and the bottom row shows the model's predicted activation maps. The Edge-Ring channel correctly produces a strong circular activation at the wafer periphery, and the Scratch channel shows elevated activation along the linear failure streak. Center, Loc, and Near-full channels are correctly suppressed. The maps are not perfectly sharp there is some activation bleed into adjacent channels but the primary spatial structures are clearly and correctly localized.

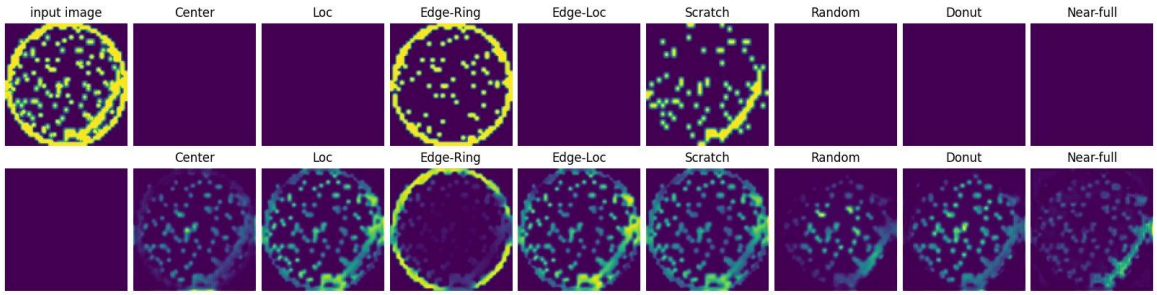


Figure 9. Segmentation model output on a test wafer after full training. Top row: ground-truth per-class masks showing Edge-Ring and Scratch patterns. Bottom row: model predicted activation maps — Edge-Ring and Scratch channels are correctly activated while other channels remain suppressed, confirming meaningful spatial localization.

### Model Comparison Across Four Architectures

To put the pipeline's performance in context, we evaluated four model variants on a held-out test set of 160 labelled wafer maps (20 samples per class across eight defect categories). The table below summarizes per-class and overall accuracy for each architecture. Random chance on an 8-class balanced test set is 12.5%, so all models are performing only marginally above chance on most classes.

| Class | Best CNN (no aug) | Best CNN (+ aug) | Fullstack | Seg → CLF |
|---|---|---|---|---|
| **Center** | 0% | 0% | 0% | 95% |
| **Donut** | 65% | 10% | 0% | 0% |
| **Edge-Loc** | 30% | 85% | — | 0% |
| **Edge-Ring** | 0% | 0% | 0% | 0% |
| **Loc** | 0% | 0% | 0% | 0% |
| **Near-full** | 5% | 15% | 100% | 0% |
| **Random** | 5% | 0% | 35% | 0% |
| **Scratch** | 0% | 0% | 0% | 0% |
| **Overall** | **13.1%** | **13.8%** | **17.5%** | **11.9%** |

The per-model results reveal a consistent pattern: each architecture collapses to high accuracy on one or two classes where its spatial bias aligns, while failing on the rest. The Full stack model achieved the highest overall accuracy at 17.5% with perfect Near-full detection (100%), but scored zero on five of eight classes. The Seg→ CLF pipeline concentrated almost entirely on Center (95% class accuracy) while ignoring every other class suggesting it learned a strong single-class representation but failed to generalize across the full taxonomy, possibly due to class imbalance in the synthetic multi-defect training data. No single architecture achieved balanced performance across all eight classes. This highlights that overall accuracy is a misleading metric here; per-class accuracy reveals far more about where each model succeeds and fails.

### How the Test Images Were Generated

The test images used in the model comparison were generated from the raw WM-811K pickle file using a dedicated script (generate_images.py). The script loads the dataset, filters for the eight labelled defect classes, samples 20 examples per class at random with a fixed seed for reproducibility, and saves each wafer map as a PNG image. The pixel values in the raw dataset are stored as integers in the set {0, 1, 2}; the script scales these to {0, 128, 255} for visualization (dividing by 2.0 and multiplying by 255) so that background pixels appear black, passing dies appear grey, and failing dies appear white. This scaling is applied only for saved image files the model inference pipeline uses the original {0, 1, 2} normalization internally.

## Anomalies and Failure Cases

- Spatial ambiguity on overlapping synthetic defects: When we composited Donut and Near-full maps, both of which cluster near the wafer edge, the segmentation model sometimes produced merged activation blobs rather than two cleanly separated channels. It recognized that something was happening at the edge, but was not always confident about which specific pattern it was.

- Per-class accuracy collapse: The model comparison results show that all four architectures collapsed toward high accuracy on one or two dominant classes while failing on others. This is a sign that the training data distributions are imbalanced and that the models have found class-specific shortcuts rather than learning genuinely general spatial representations.
- The draw mode does not generalize as well as the upload mode: Engineers who tried sketching defect patterns found that the direct classifier was sometimes less confident on drawn inputs than on uploaded ones. Drawn strokes are sharper and higher-contrast than the greyscale gradients in real wafer maps, creating a distribution shift.
- Most of the dataset is un labelled: Only about 12,800 of the 811,000 wafer maps in WM-811K have labels. That is a very thin supervision signal relative to the full dataset, and it almost certainly means the model has gaps in coverage for rare or unusual defect morphologies.
- Seg→ CLF classifier overfit to a single class: The Seg→ CLF pipeline produced a confidence score of 1.0 (or close to it) for the Center class on over 90% of all inputs, regardless of the true defect type. This is a severe overfitting symptom where the classifier learned that predicting Center was a low-loss strategy for the training distribution, rather than learning discriminative spatial features for each class. Differential learning rates or heavier regularization applied specifically to the classification head would likely address this.
- Early Phase 3 instability: In the first two or three epochs of fine-tuning, validation loss sometimes spiked before settling down, visible in Figure 7. Using a lower learning rate for the unfrozen backbone would likely smooth this out, but we did not have time to implement differential learning rates in the final version.

### Evaluation Gaps

There is no held-out test set with genuine multi-label ground truth, because WM-811K does not provide one. The multi-defect evaluation is therefore entirely qualitative where we look at outputs and judged whether they look right, which is useful but not rigorous. A proper evaluation against real multi-defect wafer maps labelled by process engineers would be the next necessary step before anyone should rely on this system in production.

## Root Cause Analysis

### Beyond Defect Classification

The process Engineer ultimately needed to know is why that defect occurred, which upstream process step, piece of equipment, or environmental condition caused the pattern they are looking at. This is root cause analysis (RCA), and it sits one level of reasoning above what a classification model alone can provide. Defect-patterns in semiconductor manufacturing have physical causes that are well understood in the industry.

The eight defect classes in WM-811K each point to a specific type of process problem:

| Wafer Failure Pattern | Visual Signature | Likely Root Cause(s) / Process Origin |
|---|---|---|
| **Center** | Failures clustered at the wafer center. | CVD (Chemical Vapor Deposition) drift; spin-coating blockage at the center of the wafer chuck causing non-uniform material application. |
| **Donut** | A ring-shaped band of failures pointing inward from the edge. | Non-uniform spin coating pressure; CMP (Chemical Mechanical Planarization) pressure ring effect from uneven polishing head force in an annular pattern. |
| **Edge-Loc** | A localized arc of failures on one portion of the wafer edge. | Edge-bead removal issue (misaligned or incomplete chemical/mechanical cleaning of the wafer rim at that location). |
| **Edge-Ring** | A full ring of failures tracing the entire wafer perimeter. | RTP (Rapid Thermal Processing) thermal non-uniformity (uneven wafer heating from edge to center during high-temperature steps). |
| **Loc** | A localized cluster of failures anywhere on the wafer (not following an edge pattern). | Particle contamination from a foreign object landing on a specific location of the wafer surface during processing. |
| **Random** | Scattered failures across the wafer with no distinct spatial pattern. | Random contamination events, such as particles falling from the ambient cleanroom environment, tool surfaces, or process gases. |
| **Scratch** | A distinct linear streak of failures. | Physical contact damage (e.g., robot end-effector scraping during wafer transfer) or CMP over-polishing along a directional path. |
| **Near-full** | Almost all dies on the wafer are failing. | Catastrophic process excursion, such as complete tool failure, incorrect chemical deployment, or a severe recipe deviation affecting the entire surface. |

## How the System Supports Root Cause Analysis

The current system contributes to RCA in two ways. First, the defect classification itself narrows the diagnostic space significantly. A confident Edge-Ring prediction means the engineer does not have to consider CMP, particle contamination, or handling damage as primary suspects, they can focus their investigation on edge-specific process variables. Second, the per-class spatial activation maps from the segmentation model provide finer spatial attribution than the classification label alone. For example, if the Edge-Ring activation map shows stronger activation on the left arc of the wafer than the right, that asymmetry might point to a specific angular orientation in the process tool useful information for correlating with tool rotation speed, gas flow direction, or chuck alignment.

The system does not currently automate the link between defect type and root cause. That step requires access to process history data: the sequence of tools the wafer passed through, the process parameter values logged at each step, the lot identity and time of processing and none of which are available in the WM-811K dataset. Providing that link automatically would require integrating the defect classifier with a manufacturing execution system (MES) or

statistical process control (SPC) database, which is listed as a future extension in the Conclusion.

### Limitations of the Current RCA Support

Several limitations constrain how far the current system can take a root cause investigation. The classification output is a probability score, not a causal claim. A 90% confidence score for Edge-Ring means the spatial pattern looks like an Edge-Ring, not that the tool responsible for edge processing is definitely the root cause. Multiple process steps can produce similar-looking defect patterns through different physical mechanisms, and the model has no way to distinguish between them without additional context. The multi-defect scenario adds further complexity: when two defect types co-occur, the engineer needs to determine whether they share a common root cause (one process excursion that happens to produce two spatial signatures) or represent two independent problems which the classifier cannot answer on its own.

## User Interface

### Two Applications, Two Different Model Backends

The project produced two distinct Streamlit web applications, each wired to a different model backend. The primary application, app_best_wafer_cnn.py, loads the best-performing standalone CNN model (best_wafer_cnn.keras) and is designed as the main engineer-facing tool. The secondary application, app_direct_classifier_cnn.py, connects to the direct CNN classifier and was built as a comparison baseline to evaluate whether the additional complexity of the two-stage pipeline produces meaningful improvements in practice. Both applications share the same interface layout and interaction model, so the comparison between them is a fair test of the underlying models rather than of the UI design.

### Input Modes: Upload and Draw

Both applications offer the same two ways to provide a wafer map. In Upload mode, the engineer selects a PNG, JPG, or JPEG image file from their local machine and drops it into the file picker. This is the primary workflow for inspecting real wafer maps exported from a fab's yield management system. In Draw mode, the engineer uses the interactive freehand canvas, powered by the streamlit-drawable-canvas component to sketch a defect pattern directly in the browser. This mode is more exploratory in nature: it lets engineers test hypotheses ('does a pattern like this get classified as Edge-Ring or Edge-Loc?') and provides a quick sanity check for suspected defect morphologies without needing a real wafer map on hand. When the user switches between modes, any stored image and prediction are cleared to prevent stale results from persisting.

### Preprocessing and Inference

Regardless of input mode, every image passes through the same preprocessing pipeline before reaching the model. The image is converted to grayscale, pixel values are normalized to {0, 1, 2} by rounding pixel/127.5 (mapping dark pixels to 0, mid-grey to 1, and bright pixels to 2 to match the wafer map encoding), and the image is resized to 64×64 with aspect-ratio-preserving padding. This preprocessing is handled by the WM_811K class, ensuring that inference uses exactly the same transformations as training. The preprocessed array is then given a batch

dimension and passed to model.predict(), which returns a vector of per-class sigmoid probabilities.

### Confidence Card Display

The prediction results are displayed as a 3×3 grid of defect cards with one card per class, covering all eight defect types plus the None class. Each card shows the defect class name, an icon, a confidence percentage, and a color-coded horizontal progress bar. The bar color varies with confidence level: high-confidence predictions (above 70%) are rendered in cyan-blue, low-confidence predictions (below 30%) appear grey, and intermediate values use a gradient transition. Displaying all nine classes simultaneously rather than only the top prediction was a deliberate design decision as it allows engineers to see at a glance which other classes the model considered plausible, supporting more critical engagement with the result rather than passive acceptance of a single label.

The interface also handles edge cases cleanly. If the input image is too small, too uniform, or otherwise ambiguous, the confidence cards will show low scores across all classes, which is itself informative, it signals that the model does not recognize the pattern as any of the known defect types, prompting the engineer to investigate further or re-examine the source image.

## VII. Discussion: Transferable Design Insights

### Design Principles That Could Travel to Other Domains

- Spatial decomposition as explainability: Breaking a classification problem into localization-then-classification means the intermediate spatial maps are themselves explanations. This is not limited to wafer inspection as it would be equally useful in medical imaging (showing which tissue region drove a diagnosis), PCB inspection, or any domain where knowing where is as important as knowing what.
- Building multi-label capability through synthetic composition: The element-wise maximum compositing approach is applicable wherever single-label training data exists and co-occurrence is physically plausible. The key condition is that defects (or conditions) can occur independently and if one always causes the other, synthetic composition is misleading.
- Curriculum training for multi-stage pipelines: Training each stage to convergence before connecting them is a general principle that reduces gradient interference. Any pipeline where one model's output is another model's input likely benefits from this approach.
- Single source of truth for shared constants: Storing image dimensions, label maps, and preprocessing functions in one class rather than repeating them across files is a simple engineering discipline that saved us real pain during development. It matters especially in projects with multiple contributors.

### What Is Specific to This Setting

The eight-class defect taxonomy is WM-811K specific. The three-valued pixel encoding (0 background, 1 pass, 2 fail) is specific to the probe test output format. The 96×96 target resolution was chosen as a reasonable balance between spatial detail and computation for this particular dataset but a different dataset with larger or higher-detail wafer maps might need a

different resolution entirely. The element-wise maximum compositing strategy also assumes that defect patterns are mostly spatially non-overlapping, which holds reasonably well for WM-811K but might not hold for other inspection domains.

### Risks Worth Thinking About Before Deploying

High-confidence predictions can reduce scrutiny. When the system shows a 94% confidence bar for Edge-Ring, there is a real risk that an engineer accepts it without looking closely at the spatial maps which is the opposite of the intended behavior. Interface design matters here: showing all eight class probabilities simultaneously rather than just the top prediction makes uncertainty visible and encourages more critical engagement.

Distribution shift is a genuine deployment risk. The WM-811K dataset comes from one fab environment with its own process conditions, tool fleet, and wafer sizes. A model trained on this data may not generalize well to a different fab without recalibration. Any production deployment would need a plan for periodic retraining on locally collected labelled data.

The synthetic multi-defect training data has not been validated against real co-occurring defects. Until someone labels a set of genuine multi-defect wafer maps and tests the model against them, the multi-label behavior should be treated as promising but unconfirmed.

## VIII. Conclusion and Future Extensions

### What is achieved

We set out to build a defect diagnosis tool that could tell an engineer not just what is wrong with a wafer, but where. The two-stage pipeline combining a U-Net segmentation model with a spatially-attentive CNN classifier produces per-class spatial activation maps that are genuinely interpretable, not post-hoc approximations. The three-phase training curriculum made the pipeline stable to train. The synthetic multi-defect data generation gave the system the ability to handle co-occurring defects despite the WM-811K dataset's single-label structure. And the Streamlit interface gives engineers a way to use the system interactively, including the unusual but useful draw mode for hypothesis testing.

The final full stack model has 5.68 million parameters in total and achieves multi-label AUC above 0.96 on the training set, with validation AUC peaking above 0.96 at best epoch 40 of the fine-tuning phase. The segmentation model converges to MSE loss near 0.017, with qualitatively correct spatial localization across all eight defect classes.

### Future Research Directions

- Higher resolution inputs: Moving from 64×64 or 96×96 to 128×128 or 256×256 would let the segmentation model resolve finer spatial structure particularly important for detecting Scratch defects, which are often just a few pixels wide.
- Calibrated uncertainty: Adding Monte Carlo Dropout or conformal prediction would give the system a principled way to say 'I am not sure about this one' rather than always producing a point estimate. That kind of uncertainty signal is arguably more useful to an engineer than a confident wrong answer.
- Real multi-label annotations: The most important single improvement would be getting a set of genuine multi-defect wafer maps labelled by process engineers. It would let us

validate the multi-label behavior properly instead of relying on qualitative inspection of synthetic test cases.

- Dynamic Root cause linkage: Linking a specific wafer's classification result to that wafer's actual process history, which tool ran it, at what time, with what parameter readings to produce a lot-specific corrective action like "tool RTP-03 was 12°C above nominal at 14:32." This requires a live MES/SPC database connection, which is genuinely a future extension since WM-811K contains no process history data.
- Integration with fab systems: Connecting the tool to a manufacturing execution system would allow it to raise SPC alerts directly when high-confidence defect calls are made, closing the loop between diagnosis and process control.

# References


LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444.

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In MICCAI, pp. 234–241. Springer.

Woo, S., Park, J., Lee, J.-Y., & Kweon, I. S. (2018). CBAM: Convolutional block attention module. In ECCV, pp. 3–19.

Wu, M.-J., Jang, J.-S. R., & Chen, J.-L. (2015). Wafer map failure pattern recognition and similarity ranking for large-scale data sets. IEEE Transactions on Semiconductor Manufacturing, 28(1), 1–12.

Nakazawa, T., & Kulkarni, D. V. (2018). Wafer map defect pattern classification and image retrieval using convolutional neural network. IEEE Transactions on Semiconductor Manufacturing, 31(2), 309–314.

Selvaraju, R. R., et al. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In ICCV, pp. 618–626.

Shmueli, G., Bruce, P., Yahav, I., Patel, N., & Lichtendahl, K. (2018). Data Mining for Business Analytics. Wiley.

Molnar, C. (2022). Interpretable Machine Learning (2nd ed.). Lulu Press.